SOURCE https://www.underworldcode.org/boundary-conditions-on-non-planar-boundaries/
UNDERWORLD underworld3 development @ 8b7c8b9e
LICENCE CC-BY-4.0
VERSION 1.0.0
ARTICLE UWTN 2026-016
PUBLISHED Aug 18, 2026

Free-Slip boundary conditions on curved boundaries

Louis Moresi [1]

[1] Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Free-slip boundary conditions are used to simplify the physical behaviour at a domain boundary. It may be a free-surface where the boundary deforms slightly in response to the internal flow, or it may be an interface where the boundary layer thickness is so small that it cannot be resolved at the same time as the interior flow. The simplifying assumption: ignore the changes in shape, ignore the thin boundary layer, treat the surface as impenetrable, and the tangential stresses as vanishingly small.

$$\boldsymbol{u} \cdot \hat{\boldsymbol{n}} = 0 \qquad \text{and} \qquad \hat{\boldsymbol{t}} \cdot \boldsymbol{\sigma} \cdot \hat{\boldsymbol{n}} = 0. \tag{1}$$

In the weak form, boundary tractions appear as surface integrals. Multiplying the momentum balance by a test function $\boldsymbol{w}$ and integrating by parts gives

$$\int_\Omega \boldsymbol{\sigma} : \nabla \boldsymbol{w} \, \mathrm{d}V - \int_{\partial\Omega} (\boldsymbol{\sigma} \cdot \hat{\boldsymbol{n}}) \cdot \boldsymbol{w} \, \mathrm{d}S = \int_\Omega \boldsymbol{f} \cdot \boldsymbol{w} \, \mathrm{d}V. \tag{2}$$

Drop the surface integral and you have imposed zero traction in all directions — free *everything* (a free surface), not free slip. Constrain the surface-normal degrees of freedom and the surface integral only addresses the tangential traction terms.

On a Cartesian box, the first expression in (1) constrains a single velocity component. If we hold u_x fixed on a vertical wall, the solver removes a row of unknowns, and there is no other work to be done. On a sphere, an annulus, any mesh that does not align with the coordinates, or a surface with topography, $\boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ is not a single component of the unknown — it constrains a combination of unknowns at each point, and leaves other combinations free. That has the potential to make a simplifying assumption complicated to implement.

Let's assume, for a moment, we confine ourselves to simple domains like the annulus, or a spherical shell, which are commonly used for planetary modelling. For each of these cases, there are coordinate systems, and well known forms of the differential operators that do restore the boundary condition to being a constraint in a single direction. Admittedly this requires reformulating all the equations, but for a symbolic-first code such as underworld, this is quite straightforward. This is the strategy used by CITCOMS Zhong et al. (2008). But not every domain boundary has a convenient coordinate system to follow. Even accounting for slight ellipticity introduces significant complexity in all the differential operators; anything more complicated will not have a useful coordinate reformulation.

The second condition in (1) is also worth noting. We don't generally think about this when we constrain a degree of freedom, the other one/s, left unconstrained are *natural* to the problem. They fall out as traction-free surface conditions automatically in a finite element weak form. If we cannot simply eliminate one degree of freedom at each point, how **do** we satisfy all the parts of (1) ?

1. Four possibilities

We outline four possible approaches (all of which you can try out in Underworld3). They fall into two pairs: two impose the constraint **weakly**, by adding a term to the momentum equation and letting the solution satisfy the condition to within the accuracy of that term: a direct penalty, and Nitsche's method, which differ in whether the term is consistent. Two impose it **exactly**: by construction, changing the basis so that the constraint is a component that can be struck out, or by a Lagrange multiplier, adding an equation that enforces it. The weak pair have a parameter to select that may need to be tuned for each problem and a floor they cannot go below. The exact pair are not tuneable, and they return the boundary traction as a side-effect of the solution. So does the direct penalty; Nitsche is the one that does not.

1.a. 1. A direct penalty

This could not be more simple, conceptually. We are working in a variational environment, so we just add into our equation system, a term that punishes any flow through the boundary:

$$\ldots + \kappa \int_{\partial\Omega} (\boldsymbol{u}\cdot\hat{\boldsymbol{n}})(\boldsymbol{w}\cdot\hat{\boldsymbol{n}})\,\mathrm{d}S. \tag{3}$$

κ is a single scalar. It has to conform to the scale of the problem itself, which is why the value that works is a property of the model rather than a default.

One line, no new machinery, and it works on any geometry. What we are solving is a mildly perturbed problem and it is perturbed by exactly the amount the constraint cannot be satisfied: the discrete solution sits where the penalty term balances the boundary traction and this leaves $\boldsymbol{u}\cdot\hat{\boldsymbol{n}}$ small but not zero.

Making the residual $\boldsymbol{u}\cdot\hat{\boldsymbol{n}}$ smaller means pushing harder, and pushing harder degrades the condition-number of the operator. The error is traded against the conditioning, and (discussed below), this trade-off eventually stops returning any benefit.

Underworld codes this term it as a boundary traction opposing normal flow, using the surface normal at the quadrature points (Γ) provided by PETSc:

```
G = mesh.Gamma
penalty = 10000
stokes.add_natural_bc(penalty * G.dot(v.sym) * G, "Upper")
```

The topography is the penalty term itself. That term is the traction the condition holds the wall with, so on the boundary

$$\sigma_{nn} = -\kappa\,(\boldsymbol{u}\cdot\hat{\boldsymbol{n}}), \qquad h = -\frac{\sigma_{nn} - \overline{\sigma_{nn}}}{\Delta\rho\, g}, \tag{4}$$

which is arithmetic on values the solve already returned — nothing is differentiated and nothing is solved.

```
n = mesh.boundary_normal("Upper")
sigma_nn = -penalty * n.dot(v.sym)
```

That is a saving in cost and not in accuracy. The leak and the traction are the same quantity scaled by κ, so a coefficient too small to hold the boundary reports a traction that is short in the same proportion.

1.b. Nitsche's method

The reason the penalty is only accurate in the limit is that it is not *consistent*: substituting the true solution does not fully satisfy the equation, because the true solution is subject to a separate boundary traction the penalty form ignores. Nitsche's method (Nitsche, 1971) restores consistency by carrying that traction explicitly:

$$\ldots - \int_{\partial\Omega} (\hat{\boldsymbol{n}} \cdot \boldsymbol{\sigma}(\boldsymbol{u}) \cdot \hat{\boldsymbol{n}})(\boldsymbol{w} \cdot \hat{\boldsymbol{n}}) \, \mathrm{d}S - \int_{\partial\Omega} (\hat{\boldsymbol{n}} \cdot \boldsymbol{\sigma}(\boldsymbol{w}) \cdot \hat{\boldsymbol{n}})(\boldsymbol{u} \cdot \hat{\boldsymbol{n}}) \, \mathrm{d}S + \frac{\gamma}{h} \int_{\partial\Omega} (\boldsymbol{u} \cdot \hat{\boldsymbol{n}})(\boldsymbol{w} \cdot \hat{\boldsymbol{n}}) \, \mathrm{d}S. \tag{5}$$

The first of the three is the consistency term: it is the boundary traction the integration by parts produced, and including this makes the true solution satisfy the discrete equations exactly. The second is its transpose, which keeps the form symmetric and buys optimal convergence in L^2. The third is the penalty again, and it is still needed — but now for *stability* rather than for accuracy, and γ has a threshold set by an inverse inequality rather than being an unspecified free parameter.

This is a real improvement and it is still done through a weak imposition. The constraint holds to the accuracy of the discretisation, not to the accuracy of the arithmetic — measured below, it leaks a few parts in a thousand on a typical mesh, and the leak falls with increasing mesh resolution.

The topography has to be recovered from the solved fields, which is where Nitsche parts company with the direct penalty. Its boundary term is the penalty part *less* the consistency terms, and those are written in $\boldsymbol{\sigma}(\boldsymbol{u})$, so the traction cannot be read off without differentiating the velocity. Recover σ_{nn} that way and divide by $\Delta\rho\, g$.

1.c. A constraint equation, with a multiplier

The two strategies above add a *term* to the weak form of the equation. This approach adds an *equation*.

Carry a scalar field λ on the boundary and require, as a row of the system in its own right,

$$\int_{\partial\Omega} (\boldsymbol{u} \cdot \hat{\boldsymbol{n}} - \tilde{u}_n)\, q \, \mathrm{d}S = 0 \quad \text{for all } q, \tag{6}$$

where $\tilde{u}_n$ is the prescribed wall-normal velocity — zero for free slip, and a datum if the wall is being driven — and λ enters the momentum row as the traction $\lambda\hat{\boldsymbol{n}}$ that holds the constraint. It is a Lagrange multiplier, and the system becomes a larger saddle point: velocity, pressure, and now λ.

λ has units of stress. At convergence it *is* σ_{nn} on that boundary, so dividing by $\Delta\rho\, g$ (density contrast $\times$ gravity) is the dynamic topography.

The constraint row is exact, so unlike a penalty there is no parameter whose size decides how well it holds. Two practical things do have to be dealt with.

- λ is carried as a full-domain field but only its boundary trace means anything, so the interior degrees of freedom are constrained out of the global system in the section before the solve sees it. They are not solved for and are not stored in the $[p, \lambda]$ block.

- The $[p, \lambda]$ Schur complement is poorly conditioned on its own, so an augmented-Lagrangian term $r(\boldsymbol{u} \cdot \hat{\boldsymbol{n}} - \tilde{u}_n)\,\hat{\boldsymbol{n}}$ is added to the momentum row. It does not change what the constraint enforces, because the λ row still carries the exact constraint.

```
stokes = uw.systems.Stokes_Constrained(mesh, velocityField=v, pressureField=p)
lam = stokes.add_constraint_bc(0.0, "Upper")
stokes.solve()
```

The `Stokes_Constrained` solver carries three fields in PETSc — velocity, pressure and λ — and splits them two ways: velocity against the pair, with the volume constraint (incompressibility) and the boundary constraint grouped into a single Schur block. The boundary constraint applies to surface nodes only.

The boundary traction is an unknown of the solve, not a post-processing step

At convergence the momentum row's boundary term *is* the normal traction, so there is nothing to recover from the velocity field afterwards. The solver returns it through `traction` and, divided by $\Delta\rho g$, through `topography`.

That boundary term is the whole boundary load, $\lambda + r(\boldsymbol{u} \cdot \hat{\boldsymbol{n}} - \tilde{u}_n)$, not the multiplier alone. The second part vanishes only where the constraint row is satisfied exactly; discretely it is satisfied to the solver's tolerance, and r multiplies that residual back into the traction. With a viscosity-weighted r and a lateral viscosity contrast it can be the largest term of the result, so the two parts are not separable in practice.

1.d. Rotating the degrees of freedom

In this approach, we stop *"asking for"* the constraint and just impose it. At each constrained node, we change the coordinate basis in which the velocity unknowns are expressed, from the global Cartesian frame to the local $(\hat{\boldsymbol{n}}, \hat{\boldsymbol{t}})$ frame. In that basis "no flow through the boundary" is again a single component, and it is removed the same way it would be on a box.

Collect the per-node rotations into a block-diagonal Q, equal to the identity at every node that is not constrained. The rotated system is

$$\hat{A} = Q^T A Q, \qquad \hat{\boldsymbol{b}} = Q^T \boldsymbol{b}, \qquad \boldsymbol{u} = Q\hat{\boldsymbol{u}}, \tag{7}$$

and the wall-normal row of $\hat{A}$ is struck out. The constraint then holds to machine precision, because it is not being solved for at all.

This is the classical strategy. It is found in the early finite-element literature, for example Engelman, Sani and Gresho (Engelman et al., 1982) reviewed the alternatives and chose between them on grounds of global mass conservation. What is worth understanding is why, given that it is exact and the others are not, it is the least used of the three.

The topography is the reaction of that struck row — the force the constraint had to supply — de-smeared by the boundary mass to turn an integrated nodal load into a pointwise stress,

$$\sigma_{nn} = -M_\Gamma^{-1}(A\boldsymbol{u} - \boldsymbol{b})\,|_\Gamma, \qquad h = -\frac{\sigma_{nn} - \overline{\sigma_{nn}}}{\Delta\rho\, g}, \tag{8}$$

which is the consistent boundary flux of Zhong, Gurnis and Hulbert (Zhong et al., 1993). In Underworld3, the solver's `boundary_normal_traction()` and `dynamic_topography()` return these. Nothing is differentiated and nothing is solved: in two dimensions M_Γ is lumped and the de-smear is a division.

2. Trade-offs

Rotating the degrees of freedom leaves the discrete problem in a **mixed basis**. Interior nodes hold (u_x, u_y); constrained nodes hold (u_n, u_t). Nothing about that is difficult in itself, but everything downstream has to agree about which nodes are which.

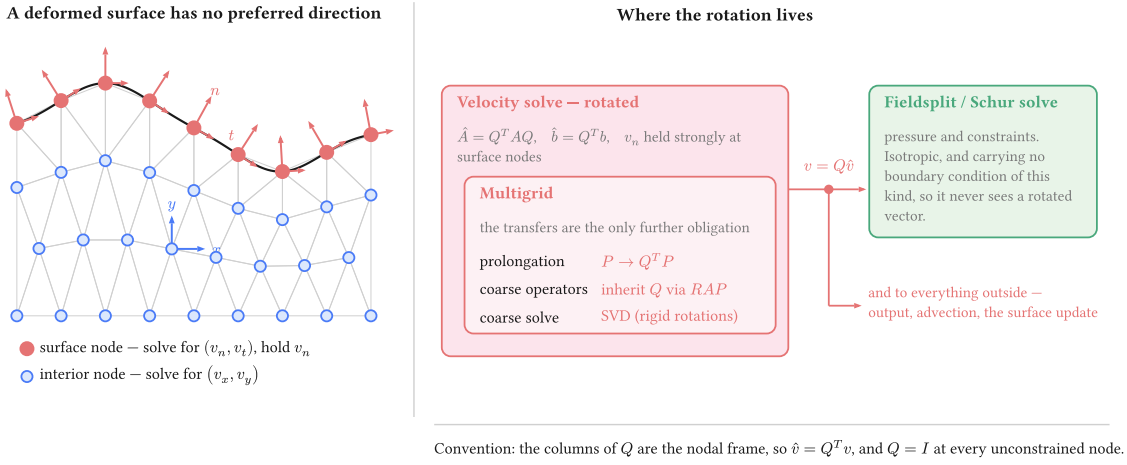

Figure 1: Where the rotation lives. The obligation is contained: the velocity solve is rotated and carries its multigrid with it, while the Schur complement and the pressure solve beside it never handle a rotated vector, because the pressure block carries no boundary condition of this kind. One un-rotation sits on the boundary between them and feeds both.

Four objects carry Q: the operator, the right-hand side, the solution on the way out, and the multigrid prolongation. The coarse operators inherit it through the Galerkin triple product rather than being rotated separately, and the coarse solve should be an SVD, because a Galerkin-coarsened rotated operator inherits any rigid-rotation null space of the constrained problem (the exact constraint makes the null space of the sphere and the annulus a dominant feature of the solve).

We do not know the cost of the addtional complexity on the solver and setup times, or on the accuracy of the solution but this can be measured and will differ from problem to problem.

3. Choice of the surface normal

In a discrete representation of a curved surface, the normal can be defined in various ways. The boundary of a discretised domain is a set of straight facets, and the assembled constraint is an integral over those facets. The node normal consistent with that integral is the average of the adjacent facet normals **weighted by facet measure** — not the normal of the smooth surface the mesh approximates, and not the facet normal on its own.**This is the consistent normal of Engelman, Sani and Gresho** (Engelman et al., 1982). They derived this result in 1982 from global conservation of mass.

The analytic normal is exact for the geometry and therefore inconsistent with the discretisation: the solver is not solving on the sphere (or annulus), it is solving on the polyhedral approximation to the sphere.

Using the **facet** normal is worse than inconsistent, it can prevent us obtaining a solution. In 2D, imposing $\boldsymbol{u} \cdot \hat{\boldsymbol{n}} = 0$ facet by facet requires a node shared by two facets to satisfy two different constraints, and two independent constraints on a two-component velocity provide no freedom. Push the penalty higher, and the vertex velocities go to zero: the flow is being asked to stay inside a polygon rather than a circle, and the discrete limit is a different problem from the smooth one. Refining the mesh does not fix this — the velocity error does not approach zero (`python3 stress.py locking`).

On an annulus with a free slip boundary, the direct-penalty approach locks at high penalty values ($\sim 10^6$) if facet normals are used in the constraint equation. In the figure below, the node-normal approach does solve and reproduces the analytic solution (described in detail in the next section)

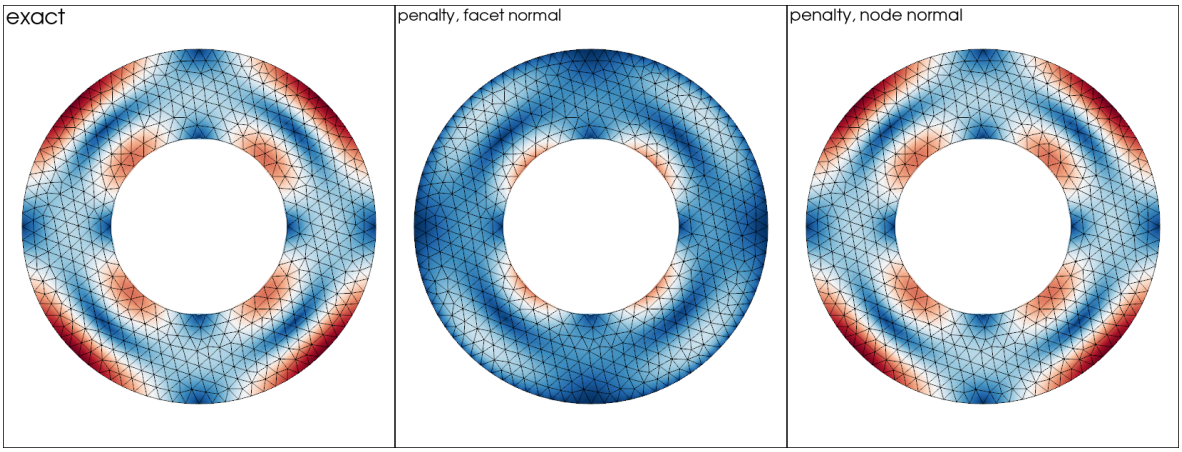

Figure 2: The same problem, the same penalty and the same scale: against the facet normal the flow along the outer boundary is suppressed — the peak speed falls by a quarter and the two fast lobes at the boundary are gone. Against the measure-weighted node normal it matches the exact solution to plotting accuracy at this resolution.

Everything that follows uses the node normal, which is what `add_nitsche_bc` and `add_rotated_freeslip_bc` take by default and what `mesh.boundary_normal` returns. The facet normal does not appear again.

4. When the choice of constraint matters

Solve a convection model with any of these approaches, and the velocity field is the same to plotting accuracy. Generally speaking, a leak of order 10^{-3} or 10^{-4} in $\boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ is within the expected accuracy of the solution on the mesh and the main driver of which method to choose should be solver efficiency (wall time).

The difference in the methods appears when the wall-normal traction is a required output of the model: dynamic topography, geoid, gravity, or a plate-boundary force balance require accurate integration of boundary stresses. Here the choice becomes more subtle, and the methods have quite different accuracies, and different efficiencies.

4.a. The benchmark

Kramer, Davies and Wilson (Kramer et al., 2021) give exact Stokes solutions in a cylindrical annulus, and their `assess` package publishes the radial stress as well as the velocity, which is what makes it an oracle for this question rather than only for the flow. Underworld wraps it as `uw.analytic.CylindricalStokes`.

The case used throughout is the smooth one: a density anomaly $(r/r_o)^k \cos n\theta$ with $n = 2$ and $k = 3$, viscosity 1, free slip on both radii. On the outer boundary the exact radial stress is a single harmonic,

$$\sigma_{rr}(r_o, \theta) = 0.1506696 \cos 2\theta, \tag{9}$$

fitted to a residual of 10^{-16}, so the whole of the surface stress is that one amplitude and the error in it is one number. The treatment under test is on the outer radius; the inner carries the exact analytic velocity as a Dirichlet condition, so it is the only free-slip condition in the model.

Two things are measured on every solve:

- **the surface permeability** — the largest $\boldsymbol{u} \cdot \hat{\boldsymbol{n}}$ on the outer boundary, against the true radial direction, divided by the flow speed. The fraction of the flow going through an impermeable boundary.
- **the boundary stress error** — the relative error in that harmonic amplitude, recovered from the solved fields by projection, which is the route every method has available.

Penalty at $\kappa = 10^4$, Nitsche at $\gamma = 10$.

cell size	penalty	Nitsche	multiplier	rotated
0.150	3.1×10^{-3} / 2.5×10^{-2}	1.0×10^{-2} / 5.8×10^{-2}	2.3×10^{-4} / 2.4×10^{-2}	5.3×10^{-11} / 2.4×10^{-2}
0.100	3.0×10^{-3} / 1.1×10^{-2}	2.4×10^{-3} / 2.4×10^{-2}	1.0×10^{-4} / 1.0×10^{-2}	5.8×10^{-11} / 1.0×10^{-2}
0.075	3.0×10^{-3} / 7.2×10^{-3}	1.2×10^{-3} / 1.5×10^{-2}	8.6×10^{-5} / 6.3×10^{-3}	1.2×10^{-10} / 6.2×10^{-3}
0.050	3.0×10^{-3} / 3.6×10^{-3}	2.7×10^{-4} / 6.3×10^{-3}	6.7×10^{-5} / 2.7×10^{-3}	1.2×10^{-10} / 2.7×10^{-3}

Reading the leak first, Nitsche leaks parts in a thousand and improves with the mesh — the rate consistency buys. The multiplier is an order of magnitude better and improves faster. The rotated constraint does not move: it sits at the solver's floor at every resolution, because the mesh has nothing to do with it. The penalty does not improve either, and for the opposite reason — its leak is set by the penalty coefficient rather than by the discretisation.

Reading the stress column, the ranking changes. **Every treatment that imposes the constraint correctly lands on the same stress error at a given mesh**: 6.3×10^{-3} for the multiplier and 6.2×10^{-3} for the rotated constraint at cell 0.075, where their leaks differ by nine orders of magnitude. What sets that number is the recovery — a projection of a stress differentiated out of a piecewise-quadratic velocity — and not the boundary condition underneath it. A constraint held to 10^{-10} buys nothing over one held to 10^{-4} if the answer is then recovered the same way.

Nitsche is the exception, at twice the error of the others on the coarser meshes. Its $\gamma = 10$ is enough for the leak and not for the stress: at $\gamma = 100$ the leak improves by a factor of nearly forty and the stress by a factor of two, onto the same floor as everything else, after which more γ buys nothing.

4.b. Traction extraction v. Stress recovery

Three of the four treatments do not have to recover anything. The two exact ones carry the traction as a constraint reaction or as an unknown, and the direct penalty carries it as the term it adds. Against the same exact answer:

cell size	rotated, reaction	multiplier, traction	penalty, $\kappa(\boldsymbol{u} \cdot \hat{\boldsymbol{n}})$	recovered by projection
0.150	6.8×10^{-3}	8.6×10^{-3}	9.0×10^{-3}	2.4×10^{-2}
0.100	3.3×10^{-3}	8.5×10^{-4}	1.2×10^{-3}	1.0×10^{-2}

cell size	rotated, reaction	multiplier, traction	penalty, $\kappa(\boldsymbol{u}\cdot\hat{\boldsymbol{n}})$	recovered by projection
0.075	2.1×10^{-3}	1.7×10^{-3}	2.3×10^{-3}	6.3×10^{-3}
0.050	1.1×10^{-3}	1.4×10^{-3}	2.2×10^{-3}	2.7×10^{-3}

Better than the projection on the same solve at every resolution, using the expressions given with each method above — by an order of magnitude at best, and by very little where the penalty's coefficient sets its floor. None of the three falls smoothly with h: part of what they report is the constraint residual, and how low a particular solve can drive that is not a function of the mesh. For the penalty that is the whole story below a cell size of 0.1 — its leak is 3×10^{-3} at every resolution here, set by κ, and the traction it reports cannot be better than its ability to satisfy the constraint.

4.c. Influence of penalty parameters

The two weak methods look alike in the comparison above, but this is for a fixed, tuned penalty parameter.

κ (penalty)	leak		γ (Nitsche)	leak
10^2	2.6×10^{-1}		1	diverged
10^3	2.6×10^{-2}		10	1.7×10^{-3}
10^4	2.6×10^{-3}		100	2.7×10^{-4}
10^5	3.0×10^{-4}		1000	3.0×10^{-5}
10^6	4.5×10^{-5}		10^4 and above	diverged

Nitsche is bounded at both ends. Below $\gamma\sim1$ the form is no longer coercive and no amount of solver tuning recovers a solution; from $\gamma\sim10^4$ in this problem, the line search stops converging. The virtue of $\gamma\sim10$ is that it sits within that window on any mesh, because γ is dimensionless and the term it scales already carries μ/h.

The penalty coefficient scales differently because it directly penalises the value of the velocity across the boundary. It should therefore scale with the characteristic velocity which is best estimated from the magnitude of the forcing terms and the resisting viscosity.

Written against the node normal, the penalty simply trades: a decade of coefficient for a decade of leak, all the way to 10^6, with no wall in this problem. What it does not do is converge with resolution — the leak is bought with the parameter rather than with the mesh resolution — so the coefficient has to be re-chosen whenever the forcing or the viscosity changes.

4.d. Multiplier and CBF equivalence

The two expressions given above for computing topography from the boundary reaction are exactly equivalent. Write the momentum row's boundary term out and the identity is immediate: the assembled load is $M_\Gamma\,(\lambda + r(\boldsymbol{u}\cdot\hat{\boldsymbol{n}} - \tilde{u}_n))$, and at convergence it balances the volume residual restricted to the boundary, which is precisely the nodal load the consistent boundary flux back-calculation reads (Zhong et al., 1993). So

$$\lambda + r(\boldsymbol{u}\cdot\hat{\boldsymbol{n}} - \tilde{u}_n) = -M_\Gamma^{-1}(A\boldsymbol{u} - \boldsymbol{b})\,|_\Gamma, \tag{10}$$

which is the rotated constraint's reaction, de-smeared with the same boundary mass. The multiplier is not an independent estimate of the surface stress: it is the same computation, arrived at by carrying the traction as an unknown instead of reading it out of the residual afterwards.

4.e. *Lateral viscosity contrast case*

No published solution has both a curved boundary and a laterally varying viscosity, so the case where weak constraints are most often reported to give trouble is a separate test with a simple geometry. SolCx is a good example: unit box, free slip on all four walls, viscosity 1 to the left of $x = 0.5$ and η_B to the right. `uw.analytic.SolCx` publishes the exact dynamic topography on the top wall. Three walls carry the ordinary component condition and the treatment under test is on the top wall alone.

On a box every treatment reduces to holding one velocity component. What it can say is whether a treatment holds the traction it was given when the viscosity beside it jumps. However, on a box the rotated constraint's per-node rotation is the identity, so the table below exercises none of the rotation machinery. We check that separately on an equivalent problem: the domain, the gravity vector and the exact solution all turned by 45°, with every wall then carrying the rotated constraint, because a component condition cannot express $\boldsymbol{u} \cdot \hat{\boldsymbol{n}} = 0$ on a tilted wall. Turned, the constraint still holds to machine precision and the velocity error is unchanged at 8.8×10^{-6}; imposing the un-turned condition on those same walls instead lets 71% of the flow through the boundary.

In the table below, we show the relative l_2 error of the surface topography along the top wall, mean removed, at 32×32 elements. Each entry is the whole wall and then the wall with two elements trimmed from each end.

η_B/η_A	component Dirichlet	penalty, 10^4	multiplier	rotated
10	0.048 / 0.054	0.045 / 0.051	0.048 / 0.054	0.048 / 0.054
10^2	0.072 / 0.081	0.056 / 0.060	0.072 / 0.081	0.072 / 0.081
10^3	0.075 / 0.084	0.234 / 0.230	0.075 / 0.084	0.075 / 0.084
10^4	0.076 / 0.085	0.698 / 0.697	0.076 / 0.085	0.076 / 0.085
10^6	0.076 / 0.085	0.992 / 1.000	0.075 / 0.084	0.076 / 0.085

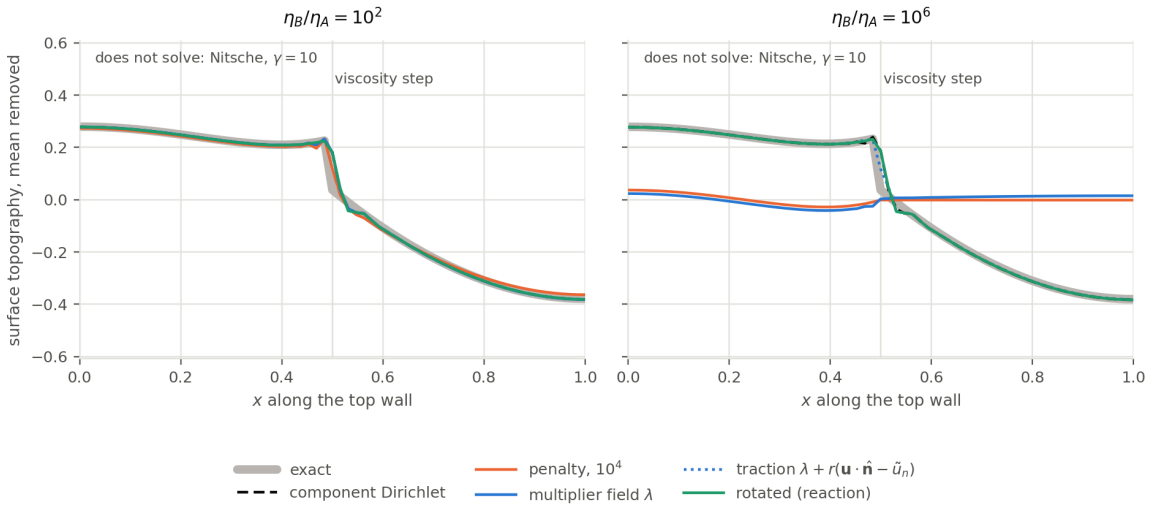

Figure 3: Surface topography along the top wall, against the exact answer. At a contrast of 100 nothing distinguishes the treatments. At 10^6 the multiplier field λ carries almost none of the traction on its own — the augmentation holds the rest — while $\lambda + r(\boldsymbol{u} \cdot \hat{\boldsymbol{n}} - \tilde{u}_n)$, which is what `traction()` returns, lies on the exact curve. The penalty has failed by this contrast: its coefficient is a bare number and cannot be large against 10^6 and moderate against 1 at the same time.

The three exact treatments agree to three figures at every contrast, whole wall and trimmed alike. That is the result to take from this half, and it took the multiplier reporting the whole traction rather than λ alone, and the rotated constraint holding the corner where it meets the side walls.

Read the first column as the reference. The component Dirichlet condition is exact and has no parameter, and its velocity error is 8.8×10^{-6} at a contrast of 10^6. It still reads 0.085. That number is the recovery's error, not a boundary condition's: on the stiff half the recovered σ_{zz} is a difference between the pressure and $2\eta\,\partial_z u_z$ with $\eta = 10^6$, so a relative velocity error of 10^{-5} appears in the stress.

A bare penalty coefficient cannot serve both halves. At 10^4 it is the best column in the table at low contrast (the constraint is weak enough not to fight the recovery) but by 10^6 it is meaningless: 0.992, which is to say the recovered topography carries none of the signal. Reading $\kappa(\boldsymbol{u}\cdot\hat{\boldsymbol{n}})$ instead of recovering the stress gives the same numbers to three figures, here and at every coefficient tried, which is the point made above from the other side: the term is the traction, so it inherits the error in the constraint rather than curing it. Scaling the coefficient by the local viscosity is the obviously right thing to want, but the solver does not converge here at any magnitude we tried, from η to $10^3\eta$.

Nitsche has no column in this table. In our implementation it is unreliable on a boundary that mixes essential patches with Nitsche patches, which is what this test asks for: the top wall weak, the other three held strongly. We could not reach a converged solution for this example at any penalty. Imposed weakly on all four walls does converge, but that is a different problem.

4.f. Solver timing

Seconds on the annulus, uniform viscosity, one core, direct solver: the solve, and then the surface traction by whatever route that treatment has. Median of three timed repeats after an untimed warm-up, run sequentially. Two sizes, because below about ten thousand nodes the four are separated by less than the run-to-run spread and there is nothing to read.

velocity nodes	penalty	Nitsche	multiplier	rotated
28 338	0.17 / —	0.18 / 0.267	0.26 / —	0.16 / 0.010
71 424	0.45 / —	0.47 / 0.657	0.67 / —	0.43 / 0.016

The dash indicates that the multiplier and the penalty do not require any additional *solver* — λ is a finite element field in its own right, so its nodal values are the traction, pointwise, and $\lambda + r(\boldsymbol{u}\cdot\hat{\boldsymbol{n}} - \tilde{u}_n)$ is an expression evaluated where it is wanted. The penalty traction is recovered similarly as the penalty scaling the leakage velocity.

The rotated constraint's reaction does need one step. The reaction is an *integrated* nodal load, $\int_\Gamma \sigma_{nn}\,\phi_i\,\mathrm{d}S$, which is M_Γ times the pointwise traction. Turning it into a pointwise value means undoing that boundary mass. On a 2-D trace, and on 3-D P1 triangles, the lumped mass is diagonal and undoing it is a division — the 10 to 16 ms above. On **3-D P2 triangles it is a true solve**: the lumped row sums vanish at the vertices, so the consistent trace mass has to be assembled and solved. That is the one place the CBF route pays for being a back-calculation.

The multiplier's solve costs about 50% more, consistently — 0.67 s against 0.43 s at 71 000 nodes. That is the extra field and the larger saddle point. Rotating the degrees of freedom costs nothing measurable against the weak forms: the rotation is a sparse orthogonal transform on a boundary's worth of rows.

Nitsche has to recover surface stress by differentiating the solution, and the projection that does it costs *more than the Stokes solve did* — 0.63 s against 0.45 s — so asking a Nitsche model for its surface stress roughly doubles the timestep. The direct penalty need not pay that: its own term is the traction, read as arithmetic on the boundary nodes.

The obvious question is whether that is the method's cost or the recovery's. A global L^2 projection to get values on a thousand boundary nodes is plainly more work than the job requires, and the consistent boundary flux is a viable alternative, reading the assembled residual rather than differentiating anything. However, we find that **it does not work for a weakly imposed condition**, and the reason is the same one that makes it unavailable to the multiplier:

cell 0.0125	projection	CBF back-calculation
penalty, node normal	0.651 s, error 1.1×10^{-3}	0.224 s, **error 1.00**
Nitsche	0.666 s, error 3.6×10^{-4}	0.230 s, **error 1.00**
rotated	0.602 s, error 1.6×10^{-4}	0.206 s, error 1.6×10^{-4}

An error of 1.00 just means that no topography was recovered. With the weak constraint, the boundary stress is recovered using the multiplier λ. In the penalty case, the boundary traction is $\kappa(\boldsymbol{u} \cdot \hat{\boldsymbol{n}})$. Nitsche's term is written in terms of $\boldsymbol{\sigma}(\boldsymbol{u})$, so reading it back still requires differentiating the answer.

That is the structural point inherent in the timing table and it follows the pairing of the strategies:

how the constraint is imposed	the traction is	to read it
weakly, by a penalty term	the term itself	evaluate $\kappa(\boldsymbol{u} \cdot \hat{\boldsymbol{n}})$
weakly, by Nitsche	inside a term written in $\boldsymbol{\sigma}(\boldsymbol{u})$	differentiate the solution
exactly, by construction (rotated)	the constraint reaction	de-smear the nodal load
exactly, by a multiplier	an unknown of the system	read the field

What these timings are not

Two-dimensional, one core, direct solver. What they measure is the difference between a recovery *solve* and boundary arithmetic, which is structural and survives scaling. They say nothing about a large parallel spherical shell, where the rotated velocity block's multigrid and the multiplier's larger Schur complement are the terms that matter and neither is exercised here.

4.g. Which one to use

For a model that consumes the velocity and nothing else, all four are the same to plotting accuracy — provided the constraint is written against the node normal. That proviso is the only one that can spoil the velocity, and it costs one line.

When the wall-normal traction is needed:

- **Rotated free slip is the default.** It holds the constraint to machine precision rather than to the discretisation, its reaction is the most accurate surface stress measured here, and that reaction is nearly free. The price is structural — a mixed basis that the multigrid has to carry — and it is paid once, inside the solver, rather than by the person setting up the model.

- **The multiplier is its equal on accuracy** and returns the same object by a different route; take it when you want the traction as an unknown of the system, or when the constrained problem's conditioning suits you better. It costs about 50% more to solve.
- **Nitsche is the one to reach for when the boundary condition must change during the model** — a wall that begins as a prescribed velocity and relaxes to a prescribed traction is a Nitsche problem, because a hard constraint cannot morph. Budget for tuning γ against the stress and not against the leak, and expect the window to move with the viscosity contrast. If the viscosity contrast is large with jumps or strong gradients along the boundary, be very careful if you choose Nitsche.
- **A direct penalty is fine for a velocity-only model** and needs the node normal, a coefficient chosen per problem, and a check on something physical before the answer is believed. It will also hand back the traction it holds the wall with, at no cost, and that reading is worth no more than the coefficient behind it. It has the advantage that this is pure, direct penalty on the weak form and can be used for many things beyond simply boundary conditions. Good for a first pass on a very general idea.

5. Using it

```
import underworld3 as uw

mesh = uw.meshing.Annulus(radiusInner=0.5, radiusOuter=1.0, cellSize=0.05)
stokes = uw.systems.Stokes(mesh)

# Value first: 0 is free slip. A non-zero scalar or expression prescribes the
# wall-normal datum u.n = u_n strongly instead.
stokes.add_rotated_freeslip_bc(0.0, "Upper")
stokes.add_rotated_freeslip_bc(0.0, "Lower")

stokes.solve()

# The constraint reaction, which is the boundary normal traction.
sigma_nn = stokes.boundary_normal_traction("Upper")
```

Leave the normal to Underworld unless the constraint has to follow the true surface rather than the mesh. Passing an analytic normal — `X / |X|` on a sphere — is exact for the geometry and keeps a consistency error against the faceted assembly, which is usually not what you want.

Reach for Nitsche when the boundary condition has to **change during the model**. A hard constraint cannot morph: a wall that begins as a prescribed velocity and relaxes to a prescribed traction is a Nitsche problem, because the rotated constraint is either imposed or it is not.

References

Engelman, M. S., Sani, R. L., & Gresho, P. M. (1982). The implementation of normal and/or tangential boundary conditions in finite element codes for incompressible fluid flow. *International Journal for Numerical Methods in Fluids*, *2*(3), 225–238. https://doi.org/10.1002/fld.1650020302

Kramer, S. C., Davies, D. R., & Wilson, C. R. (2021). Analytical solutions for mantle flow in cylindrical and spherical shells. *Geoscientific Model Development*, *14*(4), 1899–1919. https://doi.org/10.5194/gmd-14-1899-2021

Nitsche, J. (1971). Über ein Variationsprinzip zur Lösung von Dirichlet-Problemen bei Verwendung von Teilräumen, die keinen Randbedingungen unterworfen sind. *Abhandlungen Aus Dem Mathematischen Seminar Der Universität Hamburg*, *36*(1), 9–15. https://doi.org/10.1007/bf02995904

Zhong, S., Gurnis, M., & Hulbert, G. (1993). Accurate determination of surface normal stress in viscous flow from a consistent boundary flux method. *Physics of the Earth and Planetary Interiors*, *78*(1–2), 1–8. https://doi.org/10.1016/0031-9201(93)90078-n

Zhong, S., McNamara, A., Tan, E., Moresi, L., & Gurnis, M. (2008). A benchmark study on mantle convection in a 3D spherical shell using CitcomS. *Geochemistry, Geophysics, Geosystems*, *9*(10). https://doi.org/10.1029/2008gc002048